ЦИКЛЫ



Копирайт © 2026 Компас Плюс. Публичная, v1.00, 14 августа 2024 г.

Разработан: Валишина Елена.

Определение

- Цикл – оператор языка программирования, позволяющий многократно повторять одну и ту же последовательность команд.

- Вобычном SQL-запросе циклов нет. Циклы доступны только в процедурном расширении языка - PL/pgSQL: внутри анонимного блока DO, функции или процедуры.

Типы циклов

ПРОСТЫЕ

Выполняются до тех пор, пока цикл явно не закончится

FOR

Выполняются предварительно указанное число раз

WHILE

Выполняются до тех пор, пока не произойдет заданное условие

Виды циклов в PL/pgSQL

- LOOP … END LOOP – простой цикл, выход только через EXIT;
- WHILE условие LOOP – цикл с предусловием;
- FOR i IN 1..N LOOP – цикл со счётчиком;
- FOR запись IN SELECT … LOOP – цикл по результату запроса;
- FOREACH элемент IN ARRAY массив LOOP – цикл по массиву.

Отличие от Oracle

В Oracle нет цикла FOREACH и шага BY. Ассоциативных массивов (Index by BINARY_INTEGER) в PostgreSQL нет – вместо них массивы, записи таблиц и jsonb.

Типы циклов

В любом цикле обязательно должно быть задано условие выхода!

Простые циклы

```
LOOP
  операторы
  EXIT WHEN условие;
END LOOP;
```

Простые циклы

LOOP

```
DO $$
DECLARE
  vCounter numeric;
BEGIN
  vCounter := 0;
  LOOP
    vCounter := vCounter + 1;
    RAISE NOTICE 'vCounter = %', vCounter;
    EXIT WHEN vCounter = 5;
  END LOOP;
END;
$$;
```

```
NOTICE:  vCounter = 1
NOTICE:  vCounter = 2
NOTICE:  vCounter = 3
NOTICE:  vCounter = 4
NOTICE:  vCounter = 5
DO
```

Простые циклы

```
CREATE TABLE tableForTest
  (num    numeric,
   value varchar(100));
```

Простые циклы

```
INSERT INTO tableForTest
  VALUES (vCounter, vCounter * 2);
```

Простые циклы

```
DO $$
DECLARE
  vCounter numeric;
BEGIN
  vCounter := 0;
  LOOP
    vCounter := vCounter + 1;
    INSERT INTO tableForTest
      VALUES (vCounter, vCounter * 2);
    EXIT WHEN vCounter = 5;
  END LOOP;
END;
$$;

SELECT * FROM tableForTest ORDER BY num;
```

INSERT

DO

num	value
1	2
2	4
3	6
4	8
5	10

Простые циклы

```
IF vCounter = 3 THEN
  CONTINUE;
END IF;
```

Простые циклы

```
DO $$
DECLARE
  vCounter numeric;
BEGIN
  vCounter := 0;
  LOOP
    vCounter := vCounter + 1;
    IF vCounter = 3 THEN
      CONTINUE;
    END IF;
    INSERT INTO tableForTest
      VALUES (vCounter, vCounter * 2);
    EXIT WHEN vCounter = 5;
  END LOOP;
END;
$$;

SELECT * FROM tableForTest ORDER BY num;
```

CONTINUE

DO

num	value
1	2
2	4
4	8
5	10

Простые циклы

```
CONTINUE WHEN vCounter = 3;
```

Простые циклы

```
DO $$
DECLARE
  vCounter numeric;
BEGIN
  vCounter := 0;
  LOOP
    vCounter := vCounter + 1;
    CONTINUE WHEN vCounter = 3;
    INSERT INTO tableForTest
      VALUES (vCounter, vCounter * 2);
    EXIT WHEN vCounter = 5;
  END LOOP;
END;
$$;

SELECT * FROM tableForTest ORDER BY num;
```

CONTINUE WHEN

DO

num	value
1	2
2	4
4	8
5	10

Метки циклов

```
<<vOuter>>
LOOP
  ...
  EXIT vOuter WHEN условие;
END LOOP vOuter;
```

Метки циклов

```
DO $$
BEGIN
  <<vOuter>>
  FOR i IN 1..3 LOOP
    FOR j IN 1..3 LOOP
      EXIT vOuter WHEN i * j > 4;
      RAISE NOTICE 'i = %, j = %', i, j;
    END LOOP;
  END LOOP vOuter;
END;
$$;
```

МЕТКА

```
NOTICE:  i = 1, j = 1
NOTICE:  i = 1, j = 2
NOTICE:  i = 1, j = 3
NOTICE:  i = 2, j = 1
NOTICE:  i = 2, j = 2
DO
```

Метки циклов

```
CONTINUE vOuter WHEN j = 2;
```

Метки циклов

```
DO $$
BEGIN
  <<vOuter>>
  FOR i IN 1..3 LOOP
    FOR j IN 1..3 LOOP
      CONTINUE vOuter WHEN j = 2;
      RAISE NOTICE 'i = %, j = %', i, j;
    END LOOP;
  END LOOP vOuter;
END;
$$;
```

CONTINUE ПО МЕТКЕ

```
NOTICE:  i = 1, j = 1
NOTICE:  i = 2, j = 1
NOTICE:  i = 3, j = 1
DO
```

Циклы FOR

```
FOR i IN 1..5 LOOP
  операторы
END LOOP;
```

Циклы FOR

```
DO $$
BEGIN
  FOR i IN 1..5 LOOP
    RAISE NOTICE 'i = %', i;
  END LOOP;
END;
$$;
```

FOR

```
NOTICE:  i = 1
NOTICE:  i = 2
NOTICE:  i = 3
NOTICE:  i = 4
NOTICE:  i = 5
DO
```

Циклы FOR

- Счётчик цикла FOR объявлять не нужно – PL/pgSQL создаёт его автоматически и делает переменной типа integer.

- Переменная видна только внутри цикла. Если снаружи есть переменная с таким же именем, внутри цикла она недоступна.

Отличие от Oracle

В Oracle счётчик изменить нельзя: PLS-00363. В PostgreSQL присваивание разрешено, но на следующем шаге счётчик снова получает очередное значение диапазона – изменение теряется.

Циклы FOR

```
FOR i IN 1..5 LOOP
  i := i + 1;
```

Циклы FOR

```
DO $$
BEGIN
  FOR i IN 1..5 LOOP
    i := i + 1;
    RAISE NOTICE 'i = %', i;
  END LOOP;
END;
$$;
```

СЧЁТЧИК

```
NOTICE:  i = 2
NOTICE:  i = 3
NOTICE:  i = 4
NOTICE:  i = 5
NOTICE:  i = 6
DO
```

Циклы FOR

```
FOR i IN 1..5 LOOP
  INSERT INTO tableForTest
    VALUES (i, i * 3);
```

Циклы FOR

```
DO $$
BEGIN
  FOR i IN 1..5 LOOP
    INSERT INTO tableForTest
      VALUES (i, i * 3);
  END LOOP;
END;
$$;

SELECT * FROM tableForTest ORDER BY num;
```

INSERT

DO

num	value
1	3
2	6
3	9
4	12
5	15

Циклы FOR

```
FOR i IN REVERSE 1..5 LOOP
```

Циклы FOR

```
DO $$
BEGIN
  RAISE NOTICE 'начало';
  FOR i IN REVERSE 1..5 LOOP
    RAISE NOTICE 'i = %', i;
  END LOOP;
  RAISE NOTICE 'конец';
END;
$$;
```

REVERSE 1..5

```
NOTICE:  начало
NOTICE:  конец
DO
```

Циклы FOR

FOR i IN REVERSE 5..1 LOOP

Циклы FOR

```
DO $$
BEGIN
  FOR i IN REVERSE 5..1 LOOP
    RAISE NOTICE 'i = %', i;
  END LOOP;
END;
$$;
```

REVERSE 5..1

```
NOTICE:  i = 5
NOTICE:  i = 4
NOTICE:  i = 3
NOTICE:  i = 2
NOTICE:  i = 1
DO
```

REVERSE. Отличия от Oracle

	Oracle	PostgreSQL
Границы диапазона	Всегда от меньшего к большему: REVERSE 1..5	От большего к меньшему: REVERSE 5..1
REVERSE 1..5	5, 4, 3, 2, 1	Цикл не выполняется ни разу
REVERSE 5..1	Цикл не выполняется ни разу	5, 4, 3, 2, 1
Шаг	Всегда 1	Задаётся через BY

Циклы FOR

FOR i IN 1..10 BY 3 LOOP

Циклы FOR

```
DO $$
BEGIN
  FOR i IN 1..10 BY 3 LOOP
    RAISE NOTICE 'i = %', i;
  END LOOP;
END;
$$;
```

BY

```
NOTICE:  i = 1
NOTICE:  i = 4
NOTICE:  i = 7
NOTICE:  i = 10
DO
```

Цикл по запросу

- FOR запись IN SELECT … LOOP выполняет запрос и перебирает его строки. Курсор объявлять не нужно – PL/pgSQL открывает и закрывает его сам.

- Переменная цикла объявляется как record или как строка таблицы (например emp%ROWTYPE).

Отличие от Oracle

В Oracle это цикл FOR … IN (SELECT …) LOOP. В PostgreSQL скобки вокруг запроса не ставятся.

Цикл по запросу

```
FOR vEmp IN SELECT ename, job
            FROM emp
             WHERE deptno = 10 LOOP
```

Цикл по запросу

FOR IN SELECT

```
DO $$
DECLARE
  vEmp record;
BEGIN
  FOR vEmp IN SELECT ename, job
                FROM emp
               WHERE deptno = 10
               ORDER BY ename LOOP
    RAISE NOTICE '% - %', vEmp.ename, vEmp.job;
  END LOOP;
END;
$$;
```

```
NOTICE:  CLARK - MANAGER
NOTICE:  KING - PRESIDENT
NOTICE:  MILLER - CLERK
DO
```

Цикл по запросу

```
FOR vDay IN
  SELECT generate_series(vBegin, vEnd,
                         interval '1 day') LOOP
```

Цикл по запросу

GENERATE_SERIES

```
DO $$
DECLARE
  vBegin date := date_trunc('month', current_date);
  vEnd   date := date_trunc('month', current_date)
                 + interval '1 month' - interval '1 day';
  vDay   record;
  vCount numeric := 0;
BEGIN
  FOR vDay IN SELECT generate_series(vBegin, vEnd,
                                     interval '1 day') AS d LOOP
    vCount := vCount + 1;
  END LOOP;
  RAISE NOTICE 'Дней в текущем месяце: %', vCount;
END;
$$;
```

```
NOTICE:  Дней в текущем месяце: 30
DO
```

Цикл FOREACH

- FOREACH перебирает элементы массива. В Oracle такого цикла нет.

- FOREACH элемент IN ARRAY массив – перебор по одному элементу;
- FOREACH срез SLICE n IN ARRAY массив – перебор по срезам многомерного массива.

Чем заменить Index by

Массив записей – вместо таблицы записей с числовым индексом.
jsonb или hstore – вместо массива со строковым индексом.

Цикл FOREACH

```
FOREACH vName IN ARRAY vNames LOOP
  операторы
END LOOP;
```

Цикл FOREACH

```
DO $$
DECLARE
  vNames text[] := ARRAY['SMITH', 'ALLEN', 'WARD'];
  vName  text;
BEGIN
  FOREACH vName IN ARRAY vNames LOOP
    RAISE NOTICE 'vName = %', vName;
  END LOOP;
END;
$$;
```

FOREACH

```
NOTICE:  vName = SMITH
NOTICE:  vName = ALLEN
NOTICE:  vName = WARD
DO
```

Цикл FOREACH

FOREACH vRow SLICE 1 IN ARRAY vArr LOOP

Цикл FOREACH

```
DO $$
DECLARE
  vArr numeric[] := ARRAY[[1, 2, 3], [4, 5, 6]];
  vRow numeric[];
BEGIN
  FOREACH vRow SLICE 1 IN ARRAY vArr LOOP
    RAISE NOTICE 'vRow = %', vRow;
  END LOOP;
END;
$$;
```

SLICE

```
NOTICE:  vRow = {1,2,3}
NOTICE:  vRow = {4,5,6}
DO
```

Цикл FOREACH

```
vDept dept[];
SELECT array_agg(d ORDER BY d.deptno)
  INTO vDept FROM dept d;
```

Цикл FOREACH

МАССИВ ЗАПИСЕЙ

```
DO $$
DECLARE
  vDept dept[];
  vRec  dept;
BEGIN
  SELECT array_agg(d ORDER BY d.deptno)
    INTO vDept
    FROM dept d;

  FOREACH vRec IN ARRAY vDept LOOP
    RAISE NOTICE 'Номер отдела: %. Название отдела: %. Город: %',
                 vRec.deptno, vRec.dname, vRec.loc;
  END LOOP;
END;
$$;
```

```
NOTICE:  Номер отдела: 10. Название отдела: ACCOUNTING. Город: NEW YORK
NOTICE:  Номер отдела: 20. Название отдела: RESEARCH. Город: DALLAS
NOTICE:  Номер отдела: 30. Название отдела: SALES. Город: CHICAGO
NOTICE:  Номер отдела: 40. Название отдела: OPERATIONS. Город: BOSTON
DO
```

Вложенные структуры

```
FOR vDept IN SELECT key, value
             FROM jsonb_each(vEmp) LOOP
```

Вложенные структуры

JSONB

```
DO $$
DECLARE
  vEmp jsonb := '{"ACCOUNTING": {"7934": 1300},
                  "RESEARCH":   {"7369": 800, "7876": 1100},
                  "SALES":      {"7900": 950}}';
  vDept  record;
  vEmpl  record;
BEGIN
  FOR vDept IN SELECT key, value FROM jsonb_each(vEmp) LOOP
    FOR vEmpl IN SELECT key, value FROM jsonb_each_text(vDept.value) LOOP
      RAISE NOTICE '% - отдел %, оклад %',
                   vEmpl.key, vDept.key, vEmpl.value;
    END LOOP;
  END LOOP;
END;
$$;
```

```
NOTICE:  7900 - отдел SALES, оклад 950
NOTICE:  7369 - отдел RESEARCH, оклад 800
NOTICE:  7876 - отдел RESEARCH, оклад 1100
NOTICE:  7934 - отдел ACCOUNTING, оклад 1300
DO
```

Циклы WHILE

WHILE условие **LOOP**
операторы
END LOOP;

Циклы WHILE

```
DO $$
DECLARE
  vCounter numeric;
BEGIN
  vCounter := 0;
  WHILE vCounter < 6 LOOP
    vCounter := vCounter + 1;
  END LOOP;
  RAISE NOTICE 'vCounter = %', vCounter;
END;
$$;
```

WHILE

```
NOTICE:  vCounter = 6
DO
```

Циклы WHILE

```
cValue constant numeric := 8;
WHILE vCounter >= cValue LOOP
```

Циклы WHILE

```
DO $$
DECLARE
  cValue   constant numeric := 8;
  vCounter numeric;
  vNum     numeric;
BEGIN
  vCounter := 50;
  vNum     := 1;
  WHILE vCounter >= cValue LOOP
    INSERT INTO tableForTest VALUES (vNum, cValue);
    vCounter := vCounter - cValue;
    vNum     := vNum + 1;
  END LOOP;
  RAISE NOTICE 'vCounter = %', vCounter;
END;
$$;

SELECT * FROM tableForTest ORDER BY num;
```

КОНСТАНТА

```
NOTICE:  vCounter = 2
DO
```

num	value
1	8
2	8
3	8
4	8
5	8
6	8

Циклы WHILE

```
WHILE strpos(vText, '  ') <> 0 LOOP
  vText := replace(vText, '  ', ' ');
END LOOP;
```

Циклы WHILE

```
DO $$
DECLARE
  vText varchar(100);
BEGIN
  vText := 'Это просто   тест    цикла!';
  WHILE strpos(vText, '  ') <> 0 LOOP
    vText := replace(vText, '  ', ' ');
  END LOOP;
  RAISE NOTICE 'vText = %', vText;
END;
$$;
```

СТРОКИ

```
NOTICE:  vText = Это просто тест цикла!
DO
```

Циклы WHILE

```
vText := regexp_replace(vText, '\s+', ' ', 'g');
```

Циклы WHILE

БЕЗ ЦИКЛА

```
DO $$
DECLARE
  vText varchar(100);
BEGIN
  vText := 'Это просто   тест    цикла!';
  vText := regexp_replace(vText, '\s+', ' ', 'g');
  RAISE NOTICE 'vText = %', vText;
END;
$$;
```

```
NOTICE:  vText = Это просто тест цикла!
DO
```

Когда цикл не нужен

- Цикл в PL/pgSQL выполняется по одной итерации за раз, и каждая команда внутри – это отдельное обращение к серверу.

- Если ту же задачу решает один SQL-запрос или одна функция, цикл только замедляет работу:

- regexp_replace – вместо цикла с replace;
- generate_series – вместо цикла со счётчиком;
- INSERT … SELECT – вместо цикла с INSERT;
- UPDATE … FROM – вместо цикла с UPDATE.

Ловушка NULL

WHILE vCounter < 6 LOOP
-- vCounter = NULL -> цикл не выполнится ни разу

Ловушка NULL

```
DO $$
DECLARE
  vCounter numeric;
BEGIN
  WHILE vCounter < 6 LOOP
    vCounter := vCounter + 1;
  END LOOP;
  RAISE NOTICE 'vCounter = %', vCounter;
END;
$$;
```

NULL

```
NOTICE:  vCounter = <NULL>
DO
```

Принудительный выход

```
IF power(i, 2) = 144 THEN
  RAISE NOTICE '%', i;
  EXIT;
END IF;
```

Принудительный выход

```
DO $$
BEGIN
  FOR i IN 1..20 LOOP
    IF power(i, 2) = 144 THEN
      RAISE NOTICE '%', i;
      EXIT;
    END IF;
  END LOOP;
END;
$$;
```

EXIT

```
NOTICE:  12
DO
```

Зацикливание

```
WHILE power(vCount, 2) <> 132 LOOP
  vCount := vCount + 1;
END LOOP;
```

Зацикливание

```
SET statement_timeout = '5s';

DO $$
DECLARE
  vCount numeric;
BEGIN
  vCount := 0;
  WHILE power(vCount, 2) <> 132 LOOP
    vCount := vCount + 1;
  END LOOP;
  RAISE NOTICE 'vCount = %', vCount;
END;
$$;
```

БЕСКОНЕЧНЫЙ ЦИКЛ

```
ERROR:  canceling statement due to statement timeout
```

Зацикливание

```
EXIT WHEN vCount > cMaxIter;
```

Зацикливание

```
DO $$
DECLARE
  cMaxIter constant numeric := 1000;
  vCount   numeric;
BEGIN
  vCount := 0;
  WHILE power(vCount, 2) <> 132 LOOP
    vCount := vCount + 1;
    IF vCount > cMaxIter THEN
      RAISE EXCEPTION 'Превышено число итераций: %', cMaxIter;
    END IF;
  END LOOP;
END;
$$;
```

ЗАЩИТА

```
ERROR:  Превышено число итераций: 1000
```

Результат команды в цикле

- GET DIAGNOSTICS переменная = ROW_COUNT – число строк, обработанных последней командой;
- FOUND – логическая переменная: была ли обработана хотя бы одна строка.
- После цикла FOR ... IN SELECT переменная FOUND показывает, выполнился ли цикл хотя бы один раз.

Отличие от Oracle

Oracle: IF SQL%ROWCOUNT != 1 THEN EXIT; END IF;
PostgreSQL: GET DIAGNOSTICS vCount = ROW_COUNT; EXIT WHEN vCount <> 1;

Результат команды в цикле

```
GET DIAGNOSTICS vCount = ROW_COUNT;
EXIT WHEN vCount <> 1;
```

Результат команды в цикле

```
DO $$
DECLARE
  vCount integer;
BEGIN
  FOR i IN 1..10 LOOP
    UPDATE tableForTest SET value = i * 10 WHERE num = i;
    GET DIAGNOSTICS vCount = ROW_COUNT;
    RAISE NOTICE 'i = %, обновлено строк: %', i, vCount;
    EXIT WHEN vCount <> 1;
  END LOOP;
END;
$$;
```

ROW_COUNT

```
NOTICE:  i = 1, обновлено строк: 1
NOTICE:  i = 2, обновлено строк: 1
NOTICE:  i = 3, обновлено строк: 1
NOTICE:  i = 4, обновлено строк: 0
DO
```

Результат команды в цикле

```
DO $$
DECLARE
    vCount integer;
BEGIN
    UPDATE tableForTest SET value = num * 10
WHERE num > 1;
    GET DIAGNOSTICS vCount = ROW_COUNT;
    RAISE NOTICE 'update %', vCount;
END;
$$;
```

ROW_COUNT

```
NOTICE:  update 2
DO
```

Результат команды в цикле

```
FOR vEmp IN SELECT ... LOOP
END LOOP;
IF NOT FOUND THEN
```

Результат команды в цикле

```
DO $$
DECLARE
  vEmp record;
BEGIN
  FOR vEmp IN SELECT ename FROM emp WHERE deptno = 99 LOOP
    RAISE NOTICE '%', vEmp.ename;
  END LOOP;

  IF NOT FOUND THEN
    RAISE NOTICE 'Цикл не выполнился ни разу';
  END IF;
END;
$$;
```

FOUND

```
NOTICE:  Цикл не выполнился ни разу
DO
```

Производительность

```
FOR i IN 1..5 LOOP
  INSERT INTO tableForTest VALUES (i, i * 2);
END LOOP;
```

Производительность

```
DO $$
BEGIN
  FOR i IN 1..5 LOOP
    INSERT INTO tableForTest VALUES (i, i * 2);
  END LOOP;
END;
$$;

SELECT * FROM tableForTest ORDER BY num;
```

ЦИКЛ

DO

num	value
1	2
2	4
3	6
4	8
5	10

Производительность

```
INSERT INTO tableForTest
SELECT g, g * 2
  FROM generate_series(1, 5) g;
```

Производительность

```
INSERT INTO tableForTest
SELECT g, g * 2
  FROM generate_series(1, 5) g;

SELECT * FROM tableForTest ORDER BY num;
```

ОДИН ЗАПРОС

```
INSERT 0 5
```

num	value
1	2
2	4
3	6
4	8
5	10

Сравнение Oracle и PostgreSQL

Задача	Oracle	PostgreSQL
Простой цикл	LOOP … END LOOP;	LOOP … END LOOP;
Цикл с предусловием	WHILE условие LOOP	WHILE условие LOOP
Цикл со счётчиком	FOR i IN 1..5 LOOP	FOR i IN 1..5 LOOP
Обратный порядок	FOR i IN REVERSE 1..5	FOR i IN REVERSE 5..1
Шаг счётчика	Нет	FOR i IN 1..10 BY 3
Изменение счётчика	Ошибка PLS-00363	Разрешено, но теряется
Цикл по запросу	FOR r IN (SELECT …) LOOP	FOR r IN SELECT … LOOP
Цикл по массиву	Нет	FOREACH … IN ARRAY, SLICE
Ассоциативный массив	Table … Index by	Массив, jsonb, hstore
Число строк команды	SQL%ROWCOUNT	GET DIAGNOSTICS … ROW_COUNT
Вывод на экран	DBMS_OUTPUT.PUT_LINE	RAISE NOTICE

Итоги: правила работы с циклами

- В любом цикле должно быть условие выхода – иначе зацикливание.
- EXIT и CONTINUE можно писать с условием WHEN и с меткой цикла.
- В PostgreSQL REVERSE идёт от большего к меньшему: REVERSE 5..1.
- Счётчик цикла FOR не объявляют, и менять его бессмысленно.
- Для перебора строк запроса используйте FOR … IN SELECT, для массивов – FOREACH.
- Ассоциативные массивы Oracle заменяют массивы записей и jsonb.
- Число обработанных строк берите из GET DIAGNOSTICS … ROW_COUNT.
- Если задачу решает один SQL-запрос – цикл не нужен.

КОМПАС ПЛЮС

Обогащая традиции
инновациями

Спасибо!

У вас остались вопросы?





* изображение для этого слайда создано нейросетью Kandisky 2.2 по запросу «Изображение в серых тонах, которое олицетворяет синергию традиций и инноваций в финансовой сфере»